Modelos y lógica para modelar concurrencia

Programación Concurrente y Paralela, Clase práctica 7

Segundo cuatrimestre 2026

Nota. Este es el hand-out de una clase larga, con un par de cortes. Está escrito para leerse como si fuera la clase: con las mismas preguntas que haríamos en el pizarrón y en el mismo orden en que iríamos armando las ideas.

Hasta ahora, para convencernos de que un programa concurrente andaba bien hacíamos dos cosas: pensar muy fuerte y buscar trazas malas a mano. Si bien las dos sirven, tenemos algunos inconvenientes. Encontrar una traza conflictiva prueba que el programa no es correcto con respecto a alguna propiedad que queramos cumplir, pero no encontrarla no prueba nada. Bajo este problema, vamos a tratar de encarar los problemas de concurrencia bajo una óptica formal que nos ayude a verificar nuestros programas. Vamos a repasar un poco lo visto en la teórica. Primero, modelar de forma matemática un programa usando LTS. Segundo, un lenguaje para escribir lo que queremos que el programa cumpla, en una lógica modal denominada LTL. Y al final cómo juntar ambas cosas.

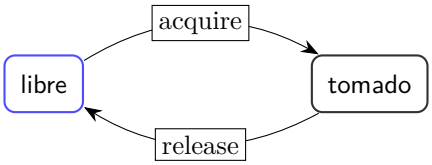
1. LTS

1.1. La definición

Recordemos:
Un *sistema de transiciones etiquetado* (LTS) es una cuádrupla $(S, A, \rightarrow, s_0)$:

- un conjunto de estados S,
- un alfabeto de acciones A,
- una relación de transición $\rightarrow \subseteq S \times A \times S$,
- y un estado inicial $s_0 \in S$.

Escribimos $s \xrightarrow{a} s'$ para decir que desde s se puede hacer la acción a y llegar a s'. Por ejemplo, un lock es un LTS de dos estados:



El dibujo es una forma cómoda de escribir la cuádrupla, pero escrito en idioma LTS, el lock sería:

- $S = \{\text{libre}, \text{tomado}\}$
- $A = \{acquire, release\}$

- $\rightarrow = \{(\text{libre}, \textit{acquire}, \text{tomado}),\ (\text{tomado}, \textit{release}, \text{libre})\}$
- $s_0 = \text{libre}$, el estado marcado en azul.

Cada flecha del dibujo es una terna de $\rightarrow$. Que no haya una flecha *acquire* desde tomado quiere decir que, con el lock tomado, *acquire* no se puede hacer. El LTS acapara todas las ejecuciones posibles de un programa, así que lo que no está ahí, no puede suceder.

Recordemos algunas definiciones que vamos a usar constantemente:

- Una **ejecución** es una sucesión de estados y acciones $s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} s_2 \cdots$ que empieza en el estado inicial y respeta las transiciones.
- Un estado es **alcanzable** si aparece en alguna ejecución. Es importante porque los LTS que vamos a dibujar, solo se ven estados alcanzables.
- Un estado sin transiciones salientes es un **deadlock**, o una terminación si el programa terminó bien. El LTS no los distingue, con lo cual es trabajo nuestro diferenciar cada caso.
- Desde un estado puede haber varias transiciones. Con lo cual, la ejecución es **no determinista**. El no determinismo representa al scheduler, que puede elegir cualquier thread para ejecutar en cualquier momento.

1.2. Composición paralela

Cuando tenemos varios LTS, los ponemos a funcionar juntos con la composición paralela. Si $P = (S_P, A_P, \rightarrow_P, p_0)$ y $Q = (S_Q, A_Q, \rightarrow_Q, q_0)$, la composición es otro LTS:

$$P \| Q = (S_P \times S_Q,\ A_P \cup A_Q,\ \rightarrow,\ (p_0, q_0))$$

Un estado es un par con el estado de cada uno, el alfabeto es la unión de los dos alfabetos y el estado inicial es el par de los iniciales. Falta definir $\rightarrow$, y para eso lo primero es mirar las acciones.

Las acciones se reparten en tres grupos. Cada acción de $A_P \cup A_Q$ cae en exactamente uno de estos:

- $A_P \setminus A_Q$: las acciones **propias** de P. Q no las conoce.
- $A_Q \setminus A_P$: las acciones propias de Q.
- $A_P \cap A_Q$: las acciones **compartidas**. Las conocen los dos, y son los puntos en los que se tienen que poner de acuerdo.

El grupo de una acción lo decide el alfabeto, no el dibujo. Una acción puede estar en A_P aunque ninguna flecha de P la tenga. Entonces P no la hace nunca, y si además está en A_Q, en la composición Q tampoco la va a poder hacer.

Una regla por grupo. La relación $\rightarrow$ tiene exactamente las transiciones que salen de estas tres reglas, y ninguna más.

$$\frac{p \xrightarrow{a}_P p' \qquad a \in A_P \setminus A_Q}{(p,q) \xrightarrow{a} (p',q)} \qquad \frac{q \xrightarrow{a}_Q q' \qquad a \in A_Q \setminus A_P}{(p,q) \xrightarrow{a} (p,q')}$$

$$\frac{p \xrightarrow{a}_P p' \qquad q \xrightarrow{a}_Q q' \qquad a \in A_P \cap A_Q}{(p,q) \xrightarrow{a} (p',q')}$$

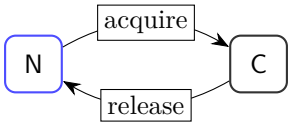
- Las dos primeras modelan el **interleave**. Una acción propia la hace su thread propio, y el otro se queda quieto en el mismo estado. Desde un par puede haber acciones propias de los dos, con lo cual, ahí aparece el no determinismo simulando al scheduler. Él decide quién avanza.
- La tercera es la **sincronización**. Una acción compartida solo se puede hacer si los dos la ofrecen en ese momento, y entonces avanzan juntos, en un único paso. Si uno no la ofrece, no pasa, aunque el otro sí la ofrezca. Cada componente bloquea las acciones de su alfabeto con solo no ofrecerlas.

Más de dos componentes. Salvo por el orden de las componentes del par, $P \| Q$ y $Q \| P$ son el mismo LTS, y lo mismo pasa con $(P \| Q) \| R$ y $P \| (Q \| R)$. Por eso escribimos $P \| Q \| R$ sin paréntesis. Para no ir armando de a dos, conviene leerlo directamente así: una acción la hacen juntos *todos* los componentes que la tienen en su alfabeto, y los demás se quedan quietos. Si alguno de los que la tienen no la ofrece, no pasa.

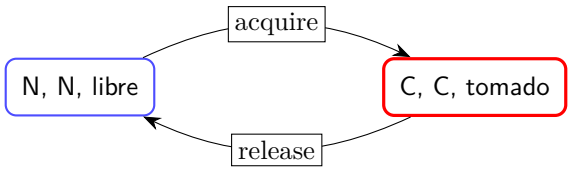
Cómo se construye. $S_P \times S_Q$ puede ser enorme, pero no hace falta mirarlo entero. Se parte de (p_0, q_0), se aplican las reglas para encontrar los sucesores, y se repite con cada estado nuevo hasta que no aparecen más. Lo que queda es justamente la parte alcanzable.

Ejemplo: dos threads y un lock. Cada thread alterna entre estar afuera de la sección crítica (N) y adentro (C), y la protege con el lock. Llamamos T_1 y T_2 a los LTS de los dos threads, y LOCK al LTS del lock de la sección 1.1, con los estados libre y tomado. Probemos primero lo más directo: los dos threads usan las mismas acciones que el lock.

T_1 y T_2

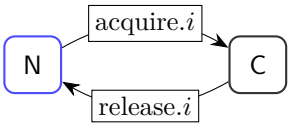


¿Qué da $\mathsf{T}_1 \| \mathsf{T}_2 \| \mathsf{LOCK}$? La acción *acquire* está en el alfabeto de los *tres*. Por la regla de sincronización, solo se puede hacer si los tres la ofrecen, y entonces avanzan los tres juntos. Desde (N, N, libre) la ofrecen los tres, así que vamos a (C, C, tomado). Y desde ahí, por lo mismo, *release* vuelve al inicio:

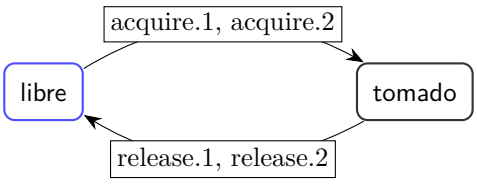


¡Los dos threads entran juntos! El modelo está mal, no el lock. En el programa, cada `acquire()` lo hace *un* thread, pero en el modelo pusimos una acción que exige a los dos. El arreglo es que la acción diga quién la hace: el thread i hace *acquire.i* y *release.i*, y el lock ofrece las de los dos.

T_i

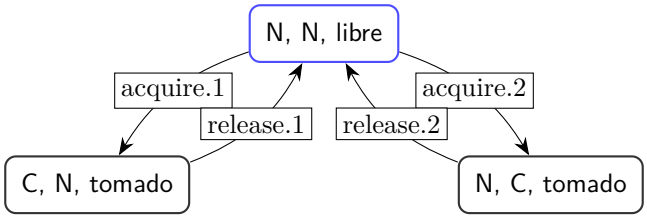


LOCK



Ahora *acquire.1* está solo en el alfabeto de T_1 y del lock. Construimos la composición desde (N, N, libre):

- *acquire.1* la ofrecen T_1 y el lock, así que vamos a (C, N, tomado). T_2 no participa, porque la acción no está en su alfabeto. Lo mismo con *acquire.2*, que lleva a (N, C, tomado).
- En (C, N, tomado), T_2 ofrece *acquire.2*, pero el lock está tomado y no la ofrece. Lo único posible es *release.1*, que vuelve al inicio.
- (N, C, tomado) es simétrico.



Son tres estados, y en ninguno están los dos threads en C. El lock garantiza la exclusión mutua porque bloquea las acciones de su alfabeto. Recordemos que el LTS modela todas las acciones, así que lo que no está ahí, no existe. Como moraleja, cuando varios componentes usan un mismo recurso, la acción tiene que decir quién la hace.

1.3. Modelar un programa, paso por paso

Empecemos por un programa con memoria compartida. Dos threads quieren reservar el mismo asiento. Cada uno se fija si está libre y, si lo está, lo reserva con su número:

```
int seat = 0     // 0: libre. Si no, el número de quien lo reservó

thread p:                          thread q:
p1: if (seat == 0)                 q1: if (seat == 0)
p2:     seat = 1                   q2:     seat = 2
```

Cada línea es atómica. ¿Puede ser que los dos threads crean que reservaron el asiento? Busquemos la respuesta a mano antes de seguir.

Vamos a construir el LTS del programa en cuatro pasos.

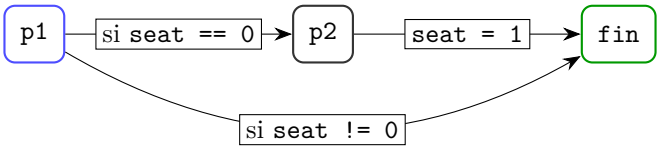
Paso 1: ¿qué es un estado? Un estado es una foto completa del sistema en un momento dado. Tiene que tener todo lo necesario para poder decidir qué puede pasar después. Acá vamos a necesitar tres cosas:

- la próxima línea de `p`, que llamamos pc_p (por program counter), con valores `p1`, `p2` o `fin`,
- la próxima línea de `q`, que llamamos pc_q,
- y el valor de `seat`.

Un estado es una terna $(pc_p, pc_q, seat)$, y el inicial es $(\texttt{p1}, \texttt{q1}, 0)$.

Acá conviene detenernos un segundo y hacernos una pregunta: ¿Hace falta algo más? Si dos situaciones del programa se pueden comportar distinto en el futuro, los modelamos como estados distintos. Ahora, si no influye en el futuro, o se puede deducir de otra cosa, no hace falta guardarlo.

Paso 2: ¿qué hace cada thread? Miramos cada thread por separado, sin pensar en el otro, y dibujamos su *diagrama de control*: por qué líneas pasa y qué efecto tiene cada una.

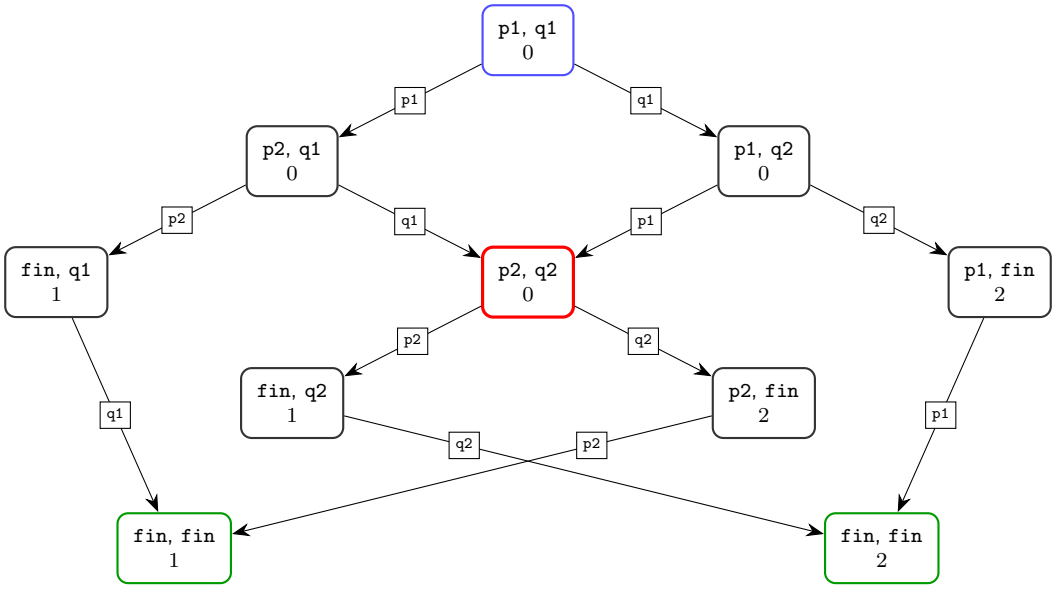


La línea `p1` tiene dos salidas, y cuál se toma depende del valor de `seat` en ese momento. Leer no cambia nada. La línea `p2` escribe. El diagrama de `q` es igual, con `q1`, `q2` y `seat = 2`.

Paso 3: la regla para avanzar. Desde un estado, *cualquiera* de los dos threads que no haya terminado puede ejecutar su próxima línea. Si avanza `p`, cambia pc_p según su diagrama, y `seat` cambia según el efecto de la línea. pc_q no cambia. Y lo mismo si avanza `q`.

Paso 4: construcción. Partimos del estado inicial y aplicamos la regla a cada estado nuevo, hasta que no aparecen más:

- $(\texttt{p1}, \texttt{q1}, 0)$: `p` lee 0 y va a `p2`, o `q` lee 0 y va a `q2`. Los sucesores son $(\texttt{p2}, \texttt{q1}, 0)$ y $(\texttt{p1}, \texttt{q2}, 0)$.
- $(\texttt{p2}, \texttt{q1}, 0)$: `p` escribe y llega a $(\texttt{fin}, \texttt{q1}, 1)$. O `q` lee, y como `seat` todavía vale 0, llega a $(\texttt{p2}, \texttt{q2}, 0)$.
- $(\texttt{p2}, \texttt{q2}, 0)$: los dos ya pasaron el `if`. Si escribe `p`, llegamos a $(\texttt{fin}, \texttt{q2}, 1)$, y si escribe `q`, a $(\texttt{p2}, \texttt{fin}, 2)$.
- $(\texttt{fin}, \texttt{q1}, 1)$: `q` lee 1 y termina sin escribir, en $(\texttt{fin}, \texttt{fin}, 1)$.
- $(\texttt{fin}, \texttt{q2}, 1)$: `q` escribe *encima* y llega a $(\texttt{fin}, \texttt{fin}, 2)$.
- Los demás son simétricos.



Cada transición está marcada con la línea que se ejecuta. Hay 10 estados alcanzables. Miremos lo que nos dice:

- En rojo está $(\texttt{p2}, \texttt{q2}, 0)$: los dos threads vieron el asiento libre y los dos van a escribir. Se llega en dos pasos, `p1` y después `q1`. Es un error típico de no sincronización.

- Desde el estado rojo, el asiento queda con el número del que escribió *último*. El otro thread cree que lo reservó, pero en realidad le pisaron el valor.
- Al estado (`fin`, `fin`, 1) se llega por dos historias distintas: una en la que solo `p` reservó, y otra en la que reservaron los dos y `q` escribió primero. El estado no recuerda cuál fue. Si la propiedad que queremos verificar dependiera de eso, habría que agregarlo al estado. Qué va en el estado depende también de qué queremos preguntar.

1.4. Ejemplo de no modelar todas las líneas

En el asiento modelamos todas las líneas y todas las variables. Muchas veces eso no hace falta, y a veces ni siquiera se puede. Veamos un vendedor que vende mientras el negocio esté abierto, y otro thread que lo cierra:

```
int sold = 0       // compartida
bool open = true   // compartida

thread seller:                      thread closer:
s1: while (open) {                  c1: open = false
s2:     int price = compute()
s3:     log("selling", price)
s4:     sold = sold + 1
    }
```

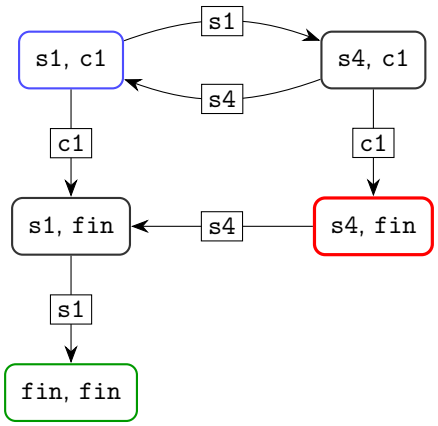
Cada línea es atómica, y `compute()` puede devolver cualquier entero. ¿Se puede vender algo después de cerrar?

Si modelamos todo. Un estado tendría pc_s, pc_c, `open`, `sold` y `price`. El problema es que `price` puede tomar cualquier valor que devuelva `compute()`, y `sold` crece sin límite mientras `closer` no se ejecute. Con lo cual el LTS es infinito, y no lo podemos dibujar. ¿Qué de todo esto influye en la respuesta?

Lo que no influye. Volvemos a la pregunta del paso 1 del anterior ejemplo. Lo que no influye en el futuro, o se puede deducir de otra cosa, no hace falta guardarlo.

- `price` es local y solo se usa en el log (una salida que nadie del programa lee). Las líneas `s2` y `s3` no tocan nada compartido, con lo cual da lo mismo que `closer` se ejecute antes o después de ellas. Las pegamos a `s4`. Después de `s1`, el próximo paso que importa es la venta.
- `sold` se escribe pero nunca se lee. Ni el `while` ni `closer` dependen de su valor, y la pregunta tampoco. La sacamos, y `s4` queda como el paso de vender, que vuelve a `s1`.
- `open` vale `false` exactamente cuando `closer` ya ejecutó `c1`. Se deduce de pc_c, así que tampoco hace falta.

Lo que nos queda es un par (pc_s, pc_c), con $pc_s \in \{\texttt{s1}, \texttt{s4}, \texttt{fin}\}$ y $pc_c \in \{\texttt{c1}, \texttt{fin}\}$. Desde `s1`, el vendedor va a `s4` si `c1` todavía no se ejecutó, y a `fin` si ya se ejecutó:



Son cinco estados. En rojo está (`s4`, `fin`). El vendedor ya pasó el chequeo y el negocio está cerrado, pero lo próximo que hace es vender. Se llega con `s1` y después `c1`. Es el mismo error que en el asiento: primero se chequea y después se actúa.

Lo que no se puede sacar es una línea que lee o escribe algo compartido. No se puede pegar a otra, porque entre las dos el otro thread puede hacer algo. Si `s1` y `s4` quedaron separadas es justamente porque entre el chequeo y la venta puede entrar `c1`. Y lo que se saca depende de la pregunta. Si quisiéramos saber cuántas ventas se registran, `sold` no se podría sacar.

1.5. ¿Etiquetas en los estados o en las transiciones?

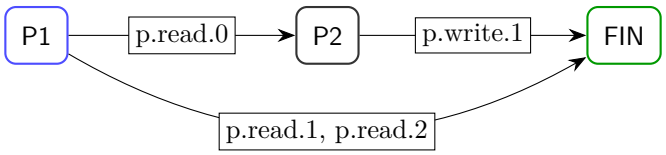
En el LTS del asiento, la información importante está en los *estados*. Cada estado dice dónde está cada thread y cuánto vale `seat`. Las transiciones están marcadas con la línea que se ejecuta, pero eso es casi decorativo. En la teórica, en cambio, la información está en las *transiciones*. Hay varios formalismos, así que conviene ordenarlos:

- Una **estructura de Kripke** tiene proposiciones en los estados y transiciones sin nombre.
- Un **LTS** tiene acciones en las transiciones y estados sin contenido. Es lo que usa FSP, y la composición se hace sincronizando por acciones.
- También hay formalismos que tienen las dos cosas a la vez, como los **sistemas de transición** (TS), con proposiciones en los estados y acciones en las transiciones.

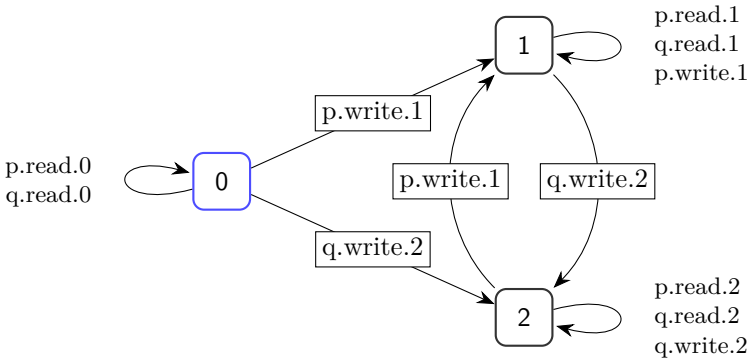
¿Cómo quedaría la reserva del asiento con etiquetas en las transiciones, al estilo de la teórica? Ahora la variable `seat` ya no es parte del estado de nadie. Pasa a ser un proceso más, SEAT, con un estado por cada valor, y se comunica con los threads por acciones:

- *p.read.v*: `p` lee el valor v. SEAT solo la ofrece si está en el estado v, y no cambia de estado.
- *p.write.1*: `p` escribe 1. SEAT la ofrece siempre y pasa al estado 1.
- Lo mismo para `q`, con *q.read.v* y *q.write.2*.

El thread `p` queda así:



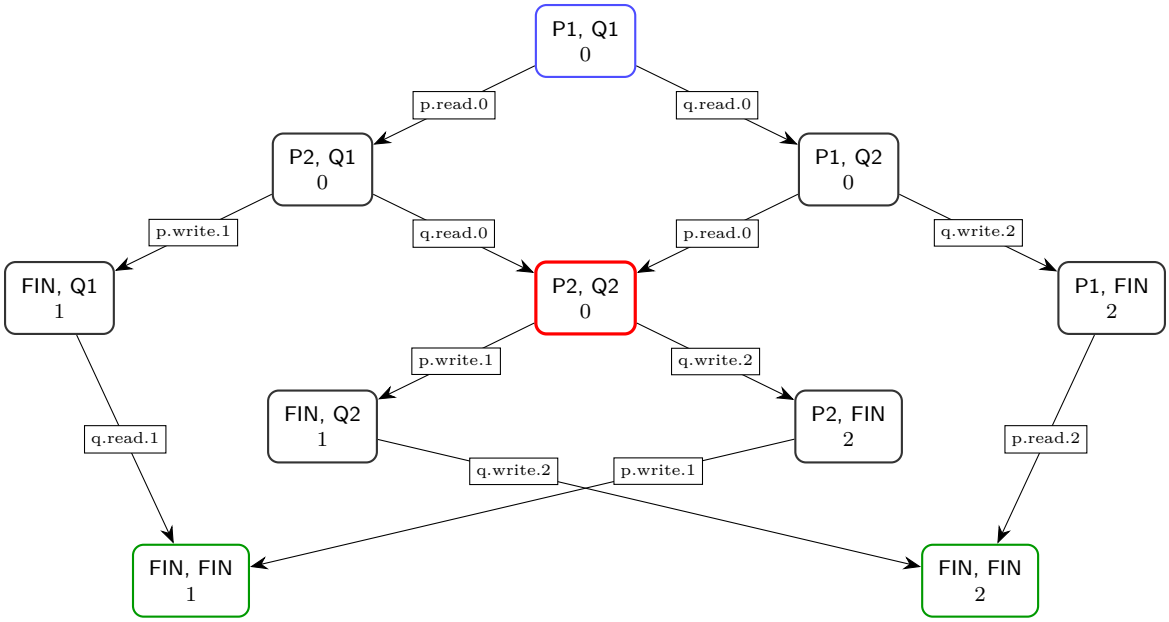
Ahora `p` no "mira" el valor de `seat`. Ofrece las tres lecturas, y SEAT bloquea las que no corresponden. Con lo cual el `if` se convirtió en una sincronización. El thread Q es igual, con *q.read.v* y *q.write.2*. Y SEAT queda así:



Los lazos son las lecturas (que no cambian el valor) y también escribir el valor que ya estaba. P y Q no comparten ninguna acción entre sí. Cada uno comparte sus acciones solo con SEAT, por eso solo se afectan a través de la variable.

Construyamos $\mathsf{P}\|\mathsf{Q}\|\mathsf{SEAT}$ desde $(\mathsf{P1}, \mathsf{Q1}, 0)$:

- P ofrece *p.read.0*, *p.read.1* y *p.read.2*, pero SEAT, en el estado 0, solo ofrece *p.read.0*. Lo mismo para Q. Entonces las únicas transiciones son *p.read.0*, que va a $(\mathsf{P2}, \mathsf{Q1}, 0)$, y *q.read.0*, que va a $(\mathsf{P1}, \mathsf{Q2}, 0)$.
- En $(\mathsf{P2}, \mathsf{Q1}, 0)$, *p.write.1* la ofrecen P y SEAT, y lleva a $(\mathsf{FIN}, \mathsf{Q1}, 1)$. *q.read.0* lleva a $(\mathsf{P2}, \mathsf{Q2}, 0)$.
- En $(\mathsf{FIN}, \mathsf{Q1}, 1)$, de las lecturas de Q, SEAT solo ofrece *q.read.1*. Por eso Q toma la rama que no escribe y termina.
- Los demás estados se construyen igual.



Es exactamente el mismo grafo de 10 estados que antes. Lo único que cambió es dónde está la información. Antes las transiciones decían qué línea se ejecutaba, y ahora dicen qué acción se sincronizó con la variable.

Las dos versiones describen lo mismo. ¿Cuál conviene? Depende de qué propiedades queramos escribir, y eso lo vamos a retomar en la sección 3, cuando ya tengamos LTL.

En la materia vamos a elegir una convención tratando de quedarnos con lo mejor de cada lado:

- Los programas con variables compartidas los modelamos como en el asiento, con el *pc* de cada thread y las variables en el estado.
- Cuando componemos LTS (por ejemplo, con pasaje de mensajes), lo hacemos por acciones, como en la teórica.

1.6. Modelar pasaje de mensajes

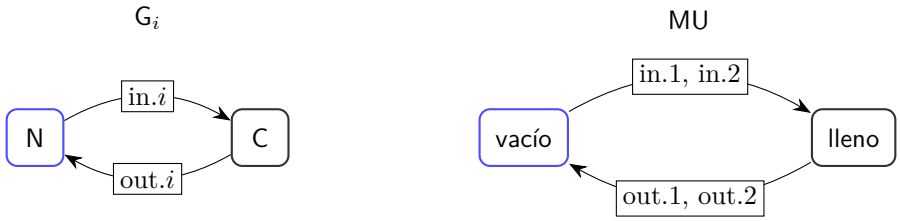
Con mensajes no hay variables compartidas, con lo cual las etiquetas en las transiciones pasan a ser lo más natural. Modelamos el envio y recibimiento de mensaje como acciones. Veamos un ejemplo en Go, un canal con buffer de capacidad uno se puede usar como lock:

```
mu := make(chan bool, 1)    // un solo lugar

// cada goroutine i, con i = 1, 2
for {
    mu <- true              // tomar: ocupar el único lugar
    critical()
    <-mu                    // soltar: liberar el lugar
}
```

Si el lugar está ocupado, el envío espera. Así, solo una goroutine puede estar entre el envío y la recepción. Modelemos el programa y veamos que es cierto.

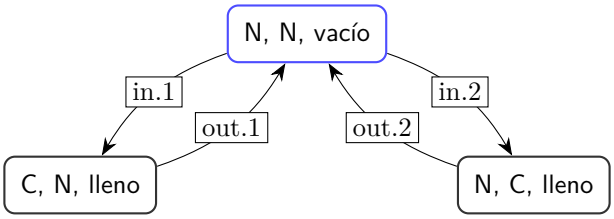
Paso 1: el LTS de cada parte. Cada goroutine alterna entre estar afuera (N) y adentro (C). Como el canal tiene buffer, mandar y recibir ya no ocurren al mismo tiempo. El envío deja el valor en el buffer y la recepción lo saca. Entre una cosa y la otra el buffer guarda algo, así que tiene que ser un LTS más, que recuerda si está vacío o lleno.



Paso 2: ¿qué acciones se comparten? Las dos goroutines mandan por el mismo canal, y pasa lo mismo que con el lock de la sección 1.2. Si las dos tuvieran una misma acción *in*, la composición las obligaría a hacerla a la vez. Por eso la acción lleva el nombre de quién la hace. *in.1* está solo en el alfabeto de G_1 y de MU, y *in.2* solo en el de G_2 y MU. El buffer ofrece las dos, y avanza con quien llegue. Lo mismo con *out.1* y *out.2*.

Paso 3: construcción. Partimos de (N, N, vacío):

- *in.1* la ofrecen G_1 y el buffer, así que vamos a (C, N, lleno). Lo mismo con *in.2*.
- En (C, N, lleno), G_2 ofrece *in.2*, pero el buffer está lleno y no la ofrece. Lo único posible es *out.1*, que vuelve al inicio.



Son tres estados, y en ninguno están las dos goroutines en C. Si cambiamos los nombres, es el mismo LTS que el de los dos threads con el lock de la sección 1.2. El canal funciona como lock porque tiene *un* lugar.

¿Qué pasa si el canal se crea sin buffer, con `make(chan bool)`? Sin buffer, mandar y recibir son una sola acción, compartida entre el que manda y el que recibe. Para que una goroutine pueda mandar, otra tiene que estar recibiendo. Pero acá cada goroutine recibe recién *después* de mandar. En el estado inicial las dos quieren mandar y ninguna recibe, con lo cual el programa está en deadlock desde el principio.

2. LTL

2.1. Trazas y posiciones

LTL habla de *trazas*. Una traza σ es una sucesión infinita de posiciones (numeradas desde 1, como en la teórica), y en cada posición vale un conjunto de proposiciones:

$$\sigma = \sigma[1]\ \sigma[2]\ \sigma[3] \cdots \qquad \text{por ejemplo} \qquad \sigma = \{p\}\ \{p, q\}\ \emptyset\ \{q\} \cdots$$

Para escribir trazas concretas nos falta notación. Recordemos la estrella de las expresiones regulares. w^* es repetir w una cantidad de veces que puede ser 0, 1, 2 o la que sea, pero siempre *finita*. Por ejemplo, $(ab)^*$ tiene a ε, ab, $abab$, $ababab$, y así. Todas son palabras finitas. Para trazas la estrella no alcanza, porque una traza es infinita, y ninguna palabra de $(\{p\}\ \emptyset)^*$ alterna para siempre. Por eso usamos un símbolo nuevo, w^ω, que es repetir w *infinitas* veces:

$$w^\omega = w\ w\ w \cdots$$

Por ejemplo, $(\{p\}\ \emptyset)^\omega$ es la traza que alterna entre $\{p\}$ y $\emptyset$ para siempre. Como la repetición no termina, la ω va siempre al final. Puede haber algo antes, como en $\emptyset\ (\{p\})^\omega$, pero nunca después.

Una fórmula se evalúa *en una posición*. Escribimos $\sigma, i \models \varphi$ para decir que φ vale en la posición i de σ. Ojo con esto, por que evaluar en la posición i no quiere decir mirar solo esa posición. Quiere decir mirar la traza *desde* i en adelante, como si empezara ahí. Por ejemplo, para saber si "siempre p" vale en la posición i, hay que mirar i, $i+1$, $i+2$, y así en adelante.

La traza satisface la fórmula si vale en la primera posición, es decir, mirando la traza entera desde el principio:

$$\sigma \models \varphi \quad \text{si y solo si} \quad \sigma, 1 \models \varphi$$

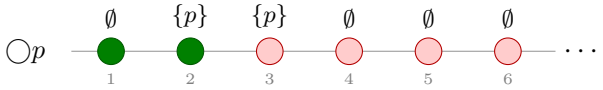
La técnica fundamental de toda esta sección es **evaluar la fórmula en cada posición**. Para una fórmula compuesta vamos de adentro hacia afuera. Primero marcamos en qué posiciones valen las partes más chicas, y con eso calculamos en qué posiciones vale la fórmula que las contiene. Es lo mismo que hacemos con las tablas de verdad, donde armamos una columna por cada subfórmula. Por ejemplo, para "siempre (p implica eventualmente q)", primero vemos dónde vale "eventualmente q", después dónde vale la implicación, y recién al final dónde vale el "siempre".

Vamos a dibujar las trazas como líneas de tiempo. Arriba de cada posición va lo que vale ahí. El color del círculo dice si la fórmula que estamos mirando vale (verde) o no (rojo) en esa posición. Cuando la línea termina en "$\cdots$", la traza sigue para siempre con el mismo patrón que se ve al final del dibujo.

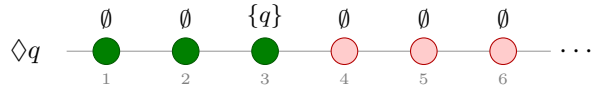
2.2. Los operadores, uno por uno

Proposiciones y conectivos. $\sigma, i \models p$ si $p \in \sigma[i]$. Los conectivos $\neg$, $\wedge$, $\vee$ y $\to$ funcionan como siempre, en la misma posición. Por ejemplo, $\sigma, i \models \varphi \wedge \psi$ si $\sigma, i \models \varphi$ y $\sigma, i \models \psi$.

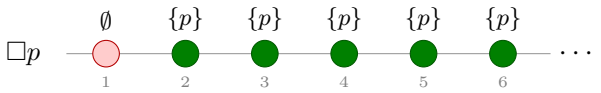
Siguiente. $\sigma, i \models \bigcirc\varphi$ si $\sigma, i+1 \models \varphi$. Mira exactamente una posición adelante.



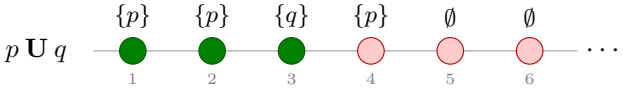
Eventualmente. $\sigma, i \models \Diamond\varphi$ si existe $j \geq i$ con $\sigma, j \models \varphi$. Ojo con el $\geq$, porque *incluye la posición actual*.



Siempre. $\sigma, i \models \Box\varphi$ si para todo $j \geq i$ vale $\sigma, j \models \varphi$. También incluye la posición actual.



Hasta. $\sigma, i \models \varphi \,\mathbf{U}\, \psi$ si existe $k \geq i$ con $\sigma, k \models \psi$, y para todo j con $i \leq j < k$ vale $\sigma, j \models \varphi$. En castellano, ψ ocurre en algún momento, y hasta entonces (sin incluirlo) vale φ.



Miremos tres cosas en este dibujo:

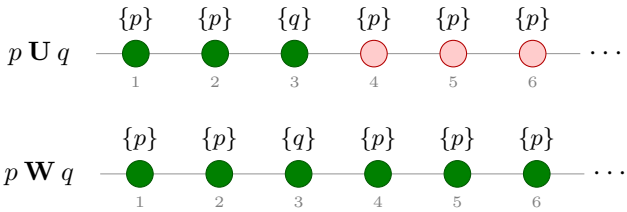
- En la posición 3 vale $p \,\mathbf{U}\, q$ porque q vale *ahí mismo*, y entonces no se le pide nada a p.
- En la posición 3 no vale p, pero no importa, porque φ se exige solo *antes* de la posición en la que vale ψ.
- En la posición 4 vale p pero no vale $p \,\mathbf{U}\, q$, porque q no vuelve a ocurrir. **U exige que ψ ocurra.**

Una forma útil de leer las definiciones es pensar a $\Box$ como un "para todo" sobre las posiciones que vienen, y a $\Diamond$ como un "existe". Muchas de las leyes que vamos a ver salen directamente de las leyes de los cuantificadores.

Hasta débil. $\sigma, i \models \varphi \,\mathbf{W}\, \psi$ si vale $\sigma, i \models \varphi \,\mathbf{U}\, \psi$, o si para todo $j \geq i$ vale $\sigma, j \models \varphi$. Escrito con los operadores que ya tenemos:

$$\varphi \,\mathbf{W}\, \psi \equiv (\varphi \,\mathbf{U}\, \psi) \vee \Box\varphi$$

En castellano, φ vale hasta que ocurra ψ, *si es que ocurre*. Si ψ no llega nunca, alcanza con que φ valga para siempre. Comparemos los dos sobre la misma traza (después de la posición 6 sigue valiendo p para siempre):



Hasta la posición 3 coinciden. Desde la 4, q no vuelve a ocurrir. **U** falla porque exige que llegue, y **W** vale porque p se sostiene para siempre.

¿Por qué existe? Porque hay muchas propiedades que piden *esperar*, pero no piden *llegar*. Pensemos en "el thread 1 no entra a la sección crítica hasta que tiene el lock". Con **U** sería $\neg \mathit{critical}_1 \,\mathbf{U}\, \mathit{acquired}_1$, pero eso además exige que el lock llegue. Con lo cual, un programa en el que el thread 1 espera para siempre la viola, aunque nunca entre sin permiso. Ahí hay dos preguntas distintas. Por un lado, si entra sin permiso. Por otro, si alguna vez consigue el lock. $\neg \mathit{critical}_1 \,\mathbf{W}\, \mathit{acquired}_1$ contesta solo la primera, que es la que queríamos.

- **Separa las dos preguntas.** $\varphi \,\mathbf{U}\, \psi \equiv (\varphi \,\mathbf{W}\, \psi) \wedge \Diamond\psi$. El **W** dice que mientras esperamos no pasa nada malo, y el $\Diamond\psi$ dice que la espera termina. También son distintas en cómo se refutan. Para refutar $\varphi \,\mathbf{W}\, \psi$ alcanza con un pedazo finito de traza, hasta una posición donde no vale ni φ ni ψ (y antes de la cual nunca valió ψ). Para refutar $\Diamond\psi$, en cambio, hay que mirar la traza entera.

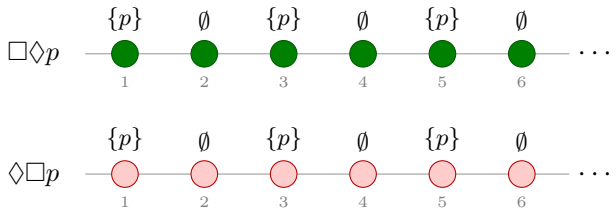
- **Generaliza a $\Box$.** $\Box\varphi \equiv \varphi \,\mathbf{W}\, \mathit{false}$. Si ψ no puede ocurrir, lo único que queda es que φ valga siempre.
- **Aparece al negar U.** $\neg(\varphi \,\mathbf{U}\, \psi) \equiv \neg\psi \,\mathbf{W}\, (\neg\varphi \wedge \neg\psi)$. Es decir, ψ no vale hasta que la espera se corta en una posición donde no vale ninguna de las dos, o ψ no vale nunca. Sin **W**, la negación de un **U** no se puede empujar hacia adentro, como vamos a hacer en las leyes con $\Box$ y $\Diamond$.

2.3. Formulas utiles para saber

Muchas propiedades se arman con unas pocas combinaciones. Conviene leerlas de afuera hacia adentro:

- $\Box\Diamond p$: "siempre, eventualmente p". Desde cualquier posición hay un p más adelante. Es decir, p **ocurre infinitas veces**.
- $\Diamond\Box p$: "eventualmente, siempre p". A partir de algún momento, p vale para siempre. p **se estabiliza**.
- $\Box(p \to \Diamond q)$: **respuesta**. Cada vez que vale p, en ese momento o después vale q.
- $\Box(p \to \Box q)$: una vez que vale p, q vale para siempre.

Para ver la diferencia entre $\Box\Diamond$ y $\Diamond\Box$, miremos la traza $(\{p\}\ \emptyset)^\omega$:



p ocurre infinitas veces, pero nunca se estabiliza. En general vale $\Diamond\Box p \to \Box\Diamond p$, pero no al revés.

2.4. Leyes

Dos fórmulas son *equivalentes*, $\varphi \equiv \psi$, si valen exactamente en las mismas posiciones de todas las trazas. No hace falta memorizar las leyes, pero sí saber que existen y entender por qué valen.

Dualidades. La negación atraviesa los operadores cambiándolos:

$$\neg \bigcirc \varphi \equiv \bigcirc\neg\varphi \qquad \neg\Diamond\varphi \equiv \Box\neg\varphi \qquad \neg\Box\varphi \equiv \Diamond\neg\varphi$$

"No es cierto que siempre φ" es lo mismo que "alguna vez no φ". Es la ley $\neg\forall \equiv \exists\neg$.

Distribución. $\Box$ distribuye sobre $\wedge$ y $\Diamond$ distribuye sobre $\vee$:

$$\Box(\varphi \wedge \psi) \equiv \Box\varphi \wedge \Box\psi \qquad \Diamond(\varphi \vee \psi) \equiv \Diamond\varphi \vee \Diamond\psi$$

Son las leyes "$\forall$ distribuye sobre $\wedge$" y "$\exists$ distribuye sobre $\vee$". Al revés *no* vale en general. $\Box$ no distribuye sobre $\vee$ y $\Diamond$ no distribuye sobre $\wedge$. $\bigcirc$, que no es un cuantificador sino una posición fija, distribuye sobre todo.

Absorción. $\Box\Box\varphi \equiv \Box\varphi$, $\Diamond\Diamond\varphi \equiv \Diamond\varphi$, $\Diamond\Box\Diamond\varphi \equiv \Box\Diamond\varphi$ y $\Box\Diamond\Box\varphi \equiv \Diamond\Box\varphi$. Cualquier tira de $\Box$ y $\Diamond$ se reduce a una de cuatro: $\Box$, $\Diamond$, $\Box\Diamond$ o $\Diamond\Box$.

Expansión. Cada operador se puede partir en "lo que pasa ahora" y "lo que queda para después":

$$\Diamond\varphi \equiv \varphi \vee \bigcirc\Diamond\varphi \qquad \Box\varphi \equiv \varphi \wedge \bigcirc\Box\varphi \qquad \varphi \,\mathbf{U}\, \psi \equiv \psi \vee (\varphi \wedge \bigcirc(\varphi \,\mathbf{U}\, \psi))$$

Por ejemplo, la última dice que, o ψ vale ahora, o vale φ ahora y el **U** queda pendiente desde la próxima posición. Estas leyes son las que usan las herramientas para construir autómatas a partir de fórmulas.

$\bigcirc$ conmuta. $\bigcirc\Box\varphi \equiv \Box\bigcirc\varphi$, $\bigcirc\Diamond\varphi \equiv \Diamond\bigcirc\varphi$ y $\bigcirc(\varphi \,\mathbf{U}\, \psi) \equiv (\bigcirc\varphi) \,\mathbf{U}\, (\bigcirc\psi)$. Da lo mismo correr todo una posición antes o después de mirar.

Definiciones cruzadas. $\Diamond\varphi \equiv \mathit{true} \,\mathbf{U}\, \varphi$ y $\Box\varphi \equiv \neg\Diamond\neg\varphi$. Así, con $\bigcirc$ y **U** alcanza para escribir todo.

2.5. Ejercicio

Dadas las trazas

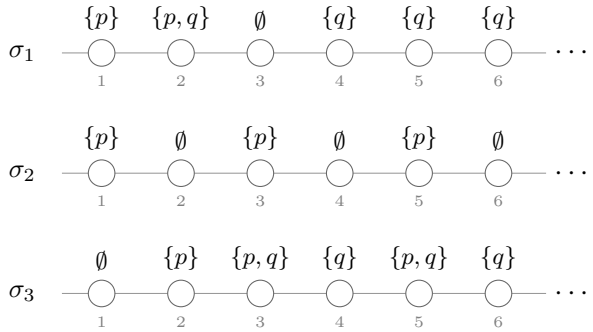
- $\sigma_1 = \{p\}\ \{p,q\}\ \emptyset\ (\{q\})^\omega$
- $\sigma_2 = (\{p\}\ \emptyset)^\omega$
- $\sigma_3 = \emptyset\ \{p\}\ (\{p,q\}\ \{q\})^\omega$

decidir cuáles satisfacen (a) $\Diamond\Box q$, (b) $\Box\Diamond p$, (c) $p \,\mathbf{U}\, q$, (d) $\bigcirc(p \,\mathbf{U}\, q)$, (e) $\Box(p \to \bigcirc q)$ y (f) $\Box(q \to \Diamond p)$.

Lo primero es escribir qué vale en cada posición:

- σ_1: en la 1 vale $\{p\}$, en la 2 vale $\{p,q\}$, en la 3 vale $\emptyset$, y de la 4 en adelante vale $\{q\}$.
- σ_2: en las impares vale $\{p\}$, y en las pares $\emptyset$.
- σ_3: en la 1 vale $\emptyset$, en la 2 vale $\{p\}$, y a partir de la 3, en las impares vale $\{p,q\}$ y en las pares $\{q\}$.

Dibujadas como líneas de tiempo, todavía sin evaluar ninguna fórmula:



(a) $\Diamond\Box q$.

- σ_1 sí, desde la posición 4.
- σ_2 no, porque q no vale nunca.
- σ_3 sí, porque desde la posición 3 vale q en todas.

(b) $\Box\Diamond p$**.**

- σ_1 no, porque p vale por última vez en la posición 2.
- σ_2 sí, porque p vale en infinitas posiciones impares.
- σ_3 sí, por lo mismo.

(c) $p \,\mathbf{U}\, q$**.** Buscamos la primera q y miramos que p valga en todas las anteriores.

- σ_1 sí, porque q vale en la posición 2 y en la 1 vale p.
- σ_2 no, porque q no vale nunca.
- σ_3 no, porque q no vale en la posición 1 y p tampoco, así que se corta enseguida.

(d) $\bigcirc(p \,\mathbf{U}\, q)$**.** Es $p \,\mathbf{U}\, q$ evaluada en la posición 2.

- σ_1 sí, porque en la posición 2 vale q.
- σ_2 no, porque q no vale nunca.
- σ_3 sí, porque en la posición 2 vale p y en la 3 vale q.

Comparemos con (c). En σ_3, correr una posición cambia la respuesta.

(e) $\Box(p \to \bigcirc q)$**.** Cada vez que vale p, en la posición siguiente vale q.

- σ_1 no, porque en la posición 2 vale p y en la 3 no vale q.
- σ_2 no, porque en la 1 vale p y en la 2 no vale q.
- σ_3 sí, porque la p de la posición 2 tiene q en la 3, y cada p en una impar tiene q en la par siguiente.

(f) $\Box(q \to \Diamond p)$**.**

- σ_1 no, porque en la posición 4 vale q y p no vuelve a valer.
- σ_2 **sí**, porque q no vale nunca, con lo cual la implicación vale en todas las posiciones (es el tercer error clásico de antes).
- σ_3 sí, porque en las impares vale p ahí mismo, y en las pares la siguiente tiene p.

2.6. Ejercicio

Para demostrar que $\varphi \equiv \psi$ hay que ver que, en *cualquier* traza y en *cualquier* posición i, si vale una vale la otra, y al revés. Para refutarla alcanza con una traza en la que una valga y la otra no. Conviene trabajar en una posición i genérica y no solo en la 1. Así la equivalencia se puede usar adentro de otras fórmulas.

(a) $\Box\Box\varphi \equiv \Box\varphi$**: vale.**

- ($\Rightarrow$) Si $\sigma, i \models \Box\Box\varphi$, entonces $\Box\varphi$ vale en todas las posiciones $j \geq i$. En particular en $j = i$.
- ($\Leftarrow$) Si $\sigma, i \models \Box\varphi$, entonces φ vale en todas las posiciones desde i. Tomemos $j \geq i$. Las posiciones desde j son parte de las posiciones desde i, así que φ vale en todas ellas, y entonces $\sigma, j \models \Box\varphi$. Como j era cualquiera, $\sigma, i \models \Box\Box\varphi$.

(b) $\Box(\varphi \vee \psi) \equiv \Box\varphi \vee \Box\psi$**: no vale.** Tomemos una traza particular. Sea $\varphi = p$ y $\psi = q$, la traza $(\{p\}\ \{q\})^\omega$ cumple $\Box(p \vee q)$, porque en cada posición vale una de las dos. Pero no cumple $\Box p$, porque en la posición 2 vale $\{q\}$ y ahí no vale p. Tampoco cumple $\Box q$, porque en la posición 1 vale $\{p\}$ y ahí no vale q. Como no cumple ninguna de las dos, no cumple $\Box p \vee \Box q$. Con lo cual la traza satisface el lado izquierdo y no el derecho. La intuición es que $\Box(\varphi \vee \psi)$ deja elegir *en cada posición* cuál de las dos vale, mientras que $\Box\varphi \vee \Box\psi$ obliga a elegir una sola para siempre. Una dirección sí vale: $\Box\varphi \vee \Box\psi \to \Box(\varphi \vee \psi)$.

(c) $\Diamond\varphi \equiv \varphi \vee \bigcirc\Diamond\varphi$**: vale.** Es la ley de expansión.

- ($\Rightarrow$) Si $\sigma, i \models \Diamond\varphi$, existe $k \geq i$ con $\sigma, k \models \varphi$. Si $k = i$, vale φ en i. Si no, $k \geq i + 1$, así que $\sigma, i + 1 \models \Diamond\varphi$, que es lo mismo que $\sigma, i \models \bigcirc\Diamond\varphi$.
- ($\Leftarrow$) Si vale φ en i, la misma posición i sirve para $\Diamond\varphi$. Si vale $\bigcirc\Diamond\varphi$ en i, hay un $k \geq i + 1$ con φ, y ese k también es $\geq i$.

(d) $\Diamond\Box\Diamond\varphi \equiv \Box\Diamond\varphi$**: vale.**

- ($\Leftarrow$) Si $\Box\Diamond\varphi$ vale en i, la posición i misma sirve de testigo para el $\Diamond$ de afuera.
- ($\Rightarrow$) Si $\sigma, i \models \Diamond\Box\Diamond\varphi$, existe $a \geq i$ tal que, para todo $j \geq a$, hay un $k \geq j$ con φ. Queremos lo mismo pero para todo $j \geq i$. Tomemos $j \geq i$ y sea m el mayor entre a y j. Como $m \geq a$, hay un $k \geq m$ con φ, y como $m \geq j$, ese k también es $\geq j$.

La intuición es que "infinitas veces φ" no depende de dónde se empiece a mirar, así que agregarle un "a partir de algún momento" no cambia nada.

2.7. Ejercicio

Una fórmula es *satisfacible* si alguna traza la satisface, y *válida* si todas la satisfacen. Para cada una, decidir:

1. $\Box\Diamond p \to \Diamond p$
2. $\bigcirc\Box p \to \Box p$
3. $\Box p \wedge \Diamond\neg p$
4. $\Box(p \to \bigcirc p) \to (p \to \Box p)$

1. Válida. Si $\Box\Diamond p$ vale en la posición 1, en particular $\Diamond p$ vale en la posición 1.

2. Satisfacible, pero no válida. $(\{p\})^\omega$ la satisface. $\emptyset\ (\{p\})^\omega$ no, porque ahí vale $\bigcirc\Box p$ pero no $\Box p$ (falla en la posición 1).

3. Insatisfacible. $\Diamond\neg p$ da una posición donde no vale p, y $\Box p$ dice que vale en todas. No hay traza que cumpla las dos. Notemos que, por la dualidad, la fórmula es $\Box p \wedge \neg\Box p$.

4. Válida. Tomemos una traza σ cualquiera. La fórmula es una implicación, así que si el lado izquierdo no vale en la posición 1, la fórmula vale y no hay nada que probar. Con lo cual podemos suponer que $\Box(p \to \bigcirc p)$ vale en la posición 1. El lado derecho también es una implicación, así que por lo mismo podemos suponer que p vale en la posición 1. Nos queda probar que vale $\Box p$ en la posición 1, es decir, que p vale en todas las posiciones $n \geq 1$. Lo hacemos por inducción en n.

- **Caso base.** En la posición 1, p vale porque lo supusimos.

- **Paso inductivo.** Supongamos que p vale en la posición n. Como $\Box(p \to \bigcirc p)$ vale en la posición 1, la implicación $p \to \bigcirc p$ vale en todas las posiciones desde la 1, y en particular en la n. Como en n vale p, también vale $\bigcirc p$. Y que $\bigcirc p$ valga en n quiere decir que p vale en $n+1$.

Entonces p vale en todas las posiciones, y vale $\Box p$.

La fórmula es exactamente el principio de inducción escrito en LTL. El p del medio es el caso base, y $\Box(p \to \bigcirc p)$ es el paso inductivo. Vale la pena ver que hacen falta las dos partes:

- Sin el caso base no alcanza. En $\emptyset^\omega$, $\Box(p \to \bigcirc p)$ vale (porque p no vale nunca, la implicación vale en todas las posiciones), pero $\Box p$ no.
- Sin el $\Box$ del paso tampoco alcanza. Si solo pidiéramos $p \to \bigcirc p$ en la posición 1, la traza $\{p\}\ \{p\}\ \emptyset^\omega$ lo cumple y empieza con p, pero p deja de valer en la posición 3.

3. LTL sobre LTS

3.1. ¿Proposiciones en los estados o en las acciones?

En la sección 1 modelamos la reserva del asiento de dos formas. Por un lado, con el *pc* de cada thread y `seat` en el estado. Por otro, con SEAT como un proceso más y la información en las acciones. Las dos dan el mismo grafo, pero no da lo mismo a la hora de escribir propiedades. "Los dos threads decidieron escribir" es algo que vale en *un estado*, el rojo. Con proposiciones en los estados es una fórmula corta, como vamos a ver enseguida. Con solo acciones, en cambio, no hay ninguna posición de la traza donde "valga" eso. Hay que describir una secuencia (hubo un *p.read.0*, después un *q.read.0*, y entre medio ninguna escritura). Se puede, pero es mucho más incómodo.

Por eso, para LTL usamos proposiciones sobre los *estados*, que calculamos a partir de lo que tiene cada estado. Por ejemplo, $critical_1$ vale en los estados en los que el thread 1 está en su sección crítica.

3.2. De un LTS a sus trazas

Para evaluar LTL sobre un LTS necesitamos trazas que las vamos a sacar de las ejecuciones. Como dijimos recién, cada estado tiene un conjunto de proposiciones que valen en él (calculado a partir de lo que tiene el estado). Una ejecución $s_1\, s_2\, s_3 \cdots$, mirando solo los estados, da la traza

$$L(s_1)\ L(s_2)\ L(s_3) \cdots$$

donde $L(s)$ es el conjunto de proposiciones que valen en s. Y entonces:

Un LTS satisface φ si *todas* sus ejecuciones, desde el estado inicial, dan trazas que satisfacen φ.

Para refutar una fórmula alcanza con una ejecución que se rompe, pero demostrarla hay que razonar sobre todas las trazas.

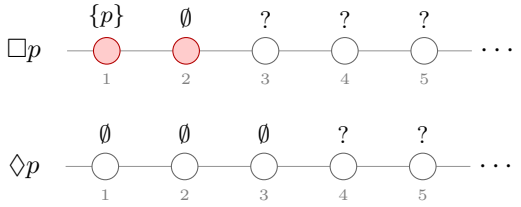
Falta un detalle. Las trazas de LTL son infinitas, pero un programa que termina tiene ejecuciones finitas. Por convención, una ejecución que llega a un estado sin salidas *se queda en ese estado para siempre*. Es decir, el último estado se repite infinitas veces.

Prefijos finitos. Para refutar una fórmula buscamos una ejecución que no la cumpla. Pero las ejecuciones son infinitas, con lo cual conviene preguntarse cuánto de la ejecución hace falta mirar. Un *prefijo* es un pedazo del principio, de largo finito. Por ejemplo, $\{p\}\ \emptyset\ \{q\}$ es un prefijo de la traza $\{p\}\ \emptyset\ \{q\}\ (\{p\})^\omega$. En un LTS, un prefijo de una ejecución es un camino finito desde el estado inicial.

Decimos que un prefijo finito **refuta** a φ si *ninguna* traza que empieza con ese prefijo satisface φ. Es decir, pase lo que pase después, la fórmula ya es falsa. Por ejemplo:

- $\{p\}\ \emptyset$ refuta a $\Box p$. En la posición 2 no vale p, y nada de lo que venga después lo arregla.
- Ningún prefijo refuta a $\Diamond p$. Cualquiera sea el prefijo, la traza podría seguir con un p más adelante, y entonces $\Diamond p$ valdría.

En las líneas de tiempo se ve así. Dibujamos solo el prefijo, y con "?" lo que todavía no sabemos cómo sigue:



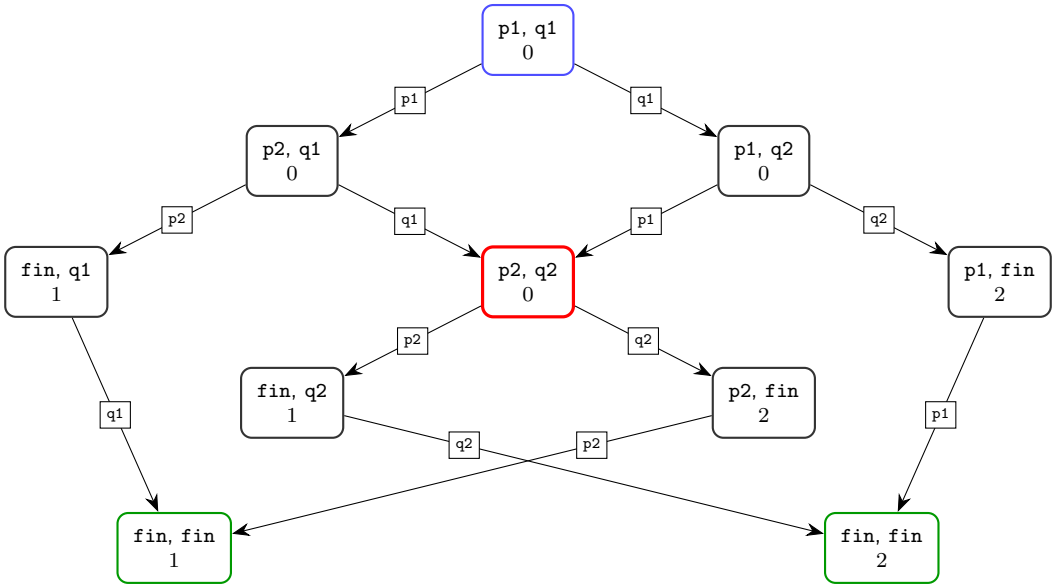
En la primera ya sabemos que $\Box p$ es falsa en las posiciones 1 y 2, siga como siga. En la segunda no podemos marcar nada, porque si más adelante aparece un p, $\Diamond p$ vale en todas esas posiciones.

Esto separa las propiedades en dos tipos, según cómo es su contraejemplo:

- **Safety.** Cada vez que fallan, hay un prefijo finito que las refuta, como $\Box p$. El contraejemplo es un camino finito desde el estado inicial hasta lo malo, y se puede mostrar entero.
- **Liveness.** Ningún prefijo finito las refuta, como $\Diamond p$. El contraejemplo tiene que ser una ejecución infinita entera. Cómo se muestra algo infinito lo vemos en el segundo ejemplo, con los *lazos*.

3.3. Ejemplo: la reserva del asiento

Volvamos al LTS de 10 estados de la sección 1. Lo repetimos acá:



¿Qué le queremos preguntar a este programa? Vamos a hacerle tres preguntas:

1. ¿Puede pasar que los dos threads crean que reservaron el asiento? Es la pregunta de la sección 1.
2. ¿El asiento termina reservado?
3. Si el asiento pasa a ser de `p`, ¿sigue siendo de `p`? O sea, ¿le pueden pisar la reserva?

Para escribirlas en LTL primero necesitamos proposiciones, y las elegimos mirando qué hace falta para cada pregunta:

- $decided_p$ vale si $pc_p = \texttt{p2}$, es decir, `p` vio el asiento libre y está por escribir. Lo mismo $decided_q$. Un thread "cree que reservó" justamente cuando pasó el `if` con el asiento libre.
- *free* vale si `seat` es 0.
- own_1 vale si `seat` es 1 (el asiento es de `p`), y own_2 si es 2.

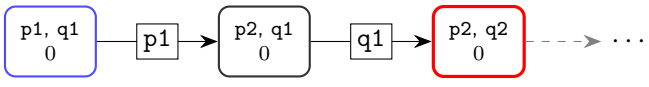
Pregunta 1: ¿los dos pueden creer que reservaron? Que los dos crean que reservaron quiere decir que los dos pasaron el `if` y están por escribir, o sea que en algún estado valen $decided_p$ y $decided_q$ a la vez. Lo que queremos es que eso no pase nunca. "Nunca" es lo mismo que "siempre no", con lo cual la propiedad es

$$\Box\neg(decided_p \wedge decided_q)$$

No vale. La ejecución

$$(\texttt{p1}, \texttt{q1}, 0)\ (\texttt{p2}, \texttt{q1}, 0)\ (\texttt{p2}, \texttt{q2}, 0) \cdots$$

llega al estado rojo, donde valen las dos. Dibujado, el contraejemplo es este camino:



Lo que viene después del estado rojo (punteado) no importa. Notemos que los tres estados del principio ya forman un prefijo finito que la refuta. Una vez que llegamos al estado rojo, la fórmula es falsa sin importar cómo siga la ejecución. Es una propiedad de safety ("nunca pasa algo malo"), y el contraejemplo es el camino hasta lo malo.

Pregunta 2: ¿el asiento termina reservado? "Termina reservado" quiere decir que a partir de algún momento `seat` deja de ser 0 y no vuelve a serlo. Ojo, no alcanza con $\Diamond\neg \mathit{free}$, que solo dice que en algún momento el asiento está reservado. Lo que queremos es que, a partir de algún momento, quede reservado para siempre:

$$\Diamond\Box\neg \mathit{free}$$

Vale. El grafo no tiene ciclos, y desde todo estado distinto de los dos finales hay alguna transición. Con lo cual, toda ejecución termina en $(\texttt{fin}, \texttt{fin}, 1)$ o en $(\texttt{fin}, \texttt{fin}, 2)$, y como son estados sin salidas, se queda ahí para siempre (es la convención que vimos en la sección 3.2 para las ejecuciones que terminan). En esos dos estados `seat` no es 0.

Pregunta 3: ¿le pueden pisar la reserva a `p`? Queremos que, cada vez que el asiento es de `p`, de ahí en adelante siga siendo de `p`. "Cada vez que" es un $\Box$ con una implicación adentro, y "de ahí en adelante" es otro $\Box$:

$$\Box(\mathit{own}_1 \to \Box \mathit{own}_1)$$

No vale. Miremos esta ejecución, que pasa por el estado rojo:

$$(\texttt{p1}, \texttt{q1}, 0)\ (\texttt{p2}, \texttt{q1}, 0)\ (\texttt{p2}, \texttt{q2}, 0)\ (\texttt{fin}, \texttt{q2}, 1)\ (\texttt{fin}, \texttt{fin}, 2) \cdots$$

En la posición 4 vale own_1, y en la 5 ya no. Al asiento le cambiaron el dueño.

3.4. Ejemplo: el canal como lock, y fair choice

Tomemos ahora el LTS de tres estados del canal usado como lock. La proposición $\mathit{critical}_i$ vale cuando la goroutine i está en C.

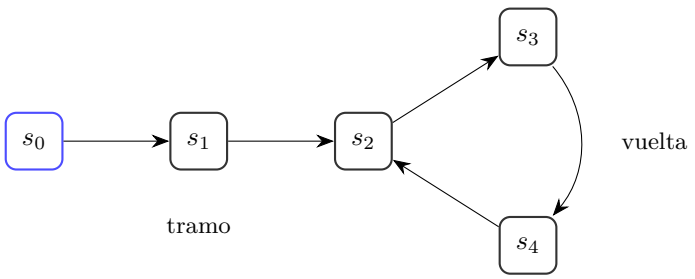
$\Box\neg(\mathit{critical}_1 \wedge \mathit{critical}_2)$: vale. En ninguno de los tres estados están las dos en C.

$\Box\Diamond \mathit{critical}_1$: no vale. Dice que la goroutine 1 entra infinitas veces. Pero la goroutine 2 puede entrar y salir para siempre:

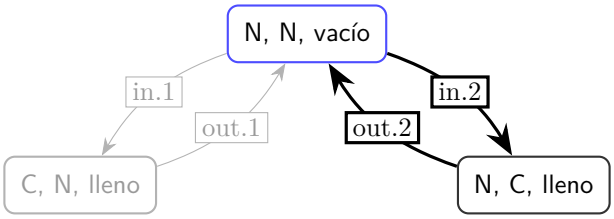
$$(\mathrm{N}, \mathrm{N}, \text{vacío})\ \big((\mathrm{N}, \mathrm{C}, \text{lleno})\ (\mathrm{N}, \mathrm{N}, \text{vacío})\big)^{\omega}$$

Este contraejemplo no puede ser finito. Ningún prefijo finito refuta a la fórmula, porque después de cualquier prefijo la goroutine 1 todavía podría entrar. Es una propiedad de liveness ("algo bueno termina pasando"), así que el contraejemplo es una ejecución infinita. ¿Cómo la escribimos, si es infinita? Acá ayuda que el LTS sea finito.

Lazos. Un **lazo** es una ejecución que tiene dos partes. Primero un *tramo* finito desde el estado inicial hasta algún estado s. Después una *vuelta* que sale de s y vuelve a s, repetida para siempre. Se llama así por la forma que tiene dibujado:



Esta ejecución se escribe $s_0\ s_1\ (s_2\ s_3\ s_4)^\omega$. Es una ejecución infinita, pero la podemos escribir con algo finito y una ω. En el canal, el tramo es solo el estado inicial, y la vuelta es ir a $(\mathrm{N}, \mathrm{C}, \text{lleno})$ y volver a $(\mathrm{N}, \mathrm{N}, \text{vacío})$. Marcado sobre el LTS (en gris lo que el lazo no usa):



La vuelta nunca pasa por $(\mathrm{C}, \mathrm{N}, \text{lleno})$, que es el único estado donde vale $critical_1$.

¿Por qué alcanza con buscar lazos? Supongamos que hay una ejecución infinita en la que, a partir de algún momento, lo bueno no pasa nunca más (acá, la goroutine 1 no vuelve a entrar). Desde ese momento la ejecución pasa por infinitos estados, pero el LTS tiene finitos, con lo cual algún estado s se repite. Tomamos como tramo el camino hasta la primera vez que aparece s después de ese momento, y como vuelta el pedazo entre esa vez y la siguiente. Repetir esa vuelta para siempre da otra ejecución, y en la vuelta lo bueno tampoco pasa. O sea, si hay un contraejemplo de liveness, hay uno con forma de lazo.

¿Es razonable este contraejemplo? Depende del scheduler. En el estado $(\mathrm{N}, \mathrm{N}, \text{vacío})$ las dos goroutines quieren entrar, y el contraejemplo supone que el scheduler elige *siempre* a la 2. La teórica formaliza la hipótesis contraria así:

> Una ejecución es válida bajo **fair choice** si, para cada estado que visita infinitas veces, toma infinitas veces cada una de sus transiciones salientes.

El lazo de arriba visita $(\mathrm{N}, \mathrm{N}, \text{vacío})$ infinitas veces y nunca toma *in.1*, con lo cual no es válido bajo fair choice. De hecho, toda ejecución infinita pasa por $(\mathrm{N}, \mathrm{N}, \text{vacío})$ una vez cada dos pasos. Bajo fair choice toma *in.1* infinitas veces, y entonces $\square\lozenge critical_1$ vale.

¿Dónde termina una ejecución justa? Supongamos que una ejecución justa se queda para siempre dando vueltas en un conjunto de estados C. Si de algún estado de C sale una transición hacia afuera de C, fair choice obliga a tomarla infinitas veces, con lo cual la ejecución no se queda en C. Entonces de C no sale ninguna transición. Además, cada transición entre estados de C se toma infinitas veces, así que la ejecución pasa por *todos* los estados de C, no solo por algunos. Llamamos **componente terminal** a un conjunto de estados en el que desde cualquiera se llega a cualquier otro, y del que no sale ninguna transición. Lo que acabamos de ver es que toda ejecución justa termina metida en una componente terminal, y adentro la recorre entera, infinitas veces.

Esto da una regla práctica. $\square\lozenge p$ vale bajo fair choice si y solo si toda componente terminal alcanzable tiene algún estado donde vale p. En el canal, los tres estados forman una sola componente terminal, y en $(\mathrm{C}, \mathrm{N}, \text{lleno})$ vale $critical_1$. Por eso $\square\lozenge critical_1$ vale. El lazo de antes no es justo justamente porque su vuelta deja afuera a $(\mathrm{C}, \mathrm{N}, \text{lleno})$.

Ahora, fair choice no ayuda con safety. Un contraejemplo de safety es un prefijo finito, y se lo puede continuar de forma justa. Fair choice solo descarta comportamientos infinitos.

3.5. Cómo demostrar o refutar, en esquema

Juntemos lo que vimos en los dos ejemplos. El punto de partida es siempre el mismo. Un LTS satisface φ si la satisfacen *todas* sus ejecuciones. Para refutar alcanza con *una* (el contraejemplo), y para demostrar hay que cubrirlas *todas*. Cómo se hace cada cosa depende del tipo de propiedad:

	Safety	**Liveness**
Qué dice	Nunca pasa algo malo.	Algo bueno termina pasando.
Formas típicas	$\Box\neg bad$, $\Box(p \to \Box q)$, $\varphi \,\mathbf{W}\, \psi$	$\Diamond p$, $\Box\Diamond p$, $\Box(p \to \Diamond q)$
Contraejemplo	Un **prefijo finito**, es decir, un camino desde el estado inicial hasta lo malo. Cómo siga después no importa.	Un **lazo**, es decir, un camino hasta un ciclo, y el ciclo repetido para siempre, sin que pase lo bueno. Un estado sin salidas cuenta como un ciclo de largo uno.
Para demostrarla	Para $\Box\neg bad$, recorremos los estados alcanzables y vemos que en ninguno vale *bad*.	Vemos que no hay ningún lazo que evite lo bueno. Para $\Box\Diamond p$, todo ciclo alcanzable, y todo estado sin salidas, tiene algún estado donde vale p.
Fair choice	No cambia nada. Un prefijo malo se puede continuar de forma justa.	Solo cuentan los lazos justos, es decir, los que dan vueltas por una componente terminal entera. Para $\Box\Diamond p$, vale si toda componente terminal alcanzable tiene algún estado donde vale p.

Para decidir de qué tipo es una propiedad, conviene hacerse una pregunta. Si se viola, ¿se puede señalar *el momento* en que se violó? Si se puede, es safety. Si para verla violada hay que mirar la ejecución entera, es liveness.

Algunas fórmulas tienen las dos partes. Por ejemplo, en $\varphi \,\mathbf{U}\, \psi \equiv (\varphi \,\mathbf{W}\, \psi) \wedge \Diamond\psi$, la primera parte es safety y la segunda liveness. En ese caso tratamos cada parte por separado, y para refutar alcanza con que falle una.

3.6. Ejercicio

Dos threads se turnan para entrar a la sección crítica con una variable compartida `turn`:

```
int turn = 1     // compartida

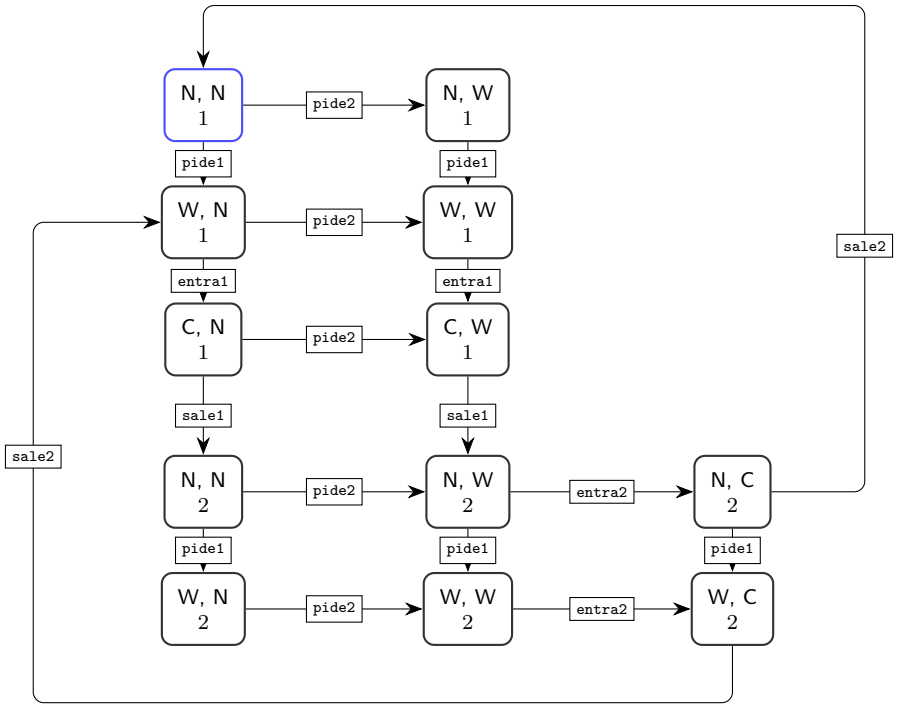
// cada thread i, con i = 1, 2, y j el otro
while (true) {
    otras_cosas()          // N
    await (turn == i)      // W: se bloquea hasta que turn sea i
    critical()             // C
    turn = j
}
```

Cada thread pasa por tres lugares. N (afuera), W (esperando su turno) y C (adentro). Al salir de C le pasa el turno al otro. Tomamos las proposiciones $wait_i$, que vale si el thread i está en W, y $critical_i$, que vale si está en C. Decidir si el programa satisface

1. $\Box\neg(critical_1 \wedge critical_2)$,
2. $\Box(wait_1 \to \Diamond critical_1)$, con y sin fair choice,

y qué pasa con la segunda si el thread 2, cada vez que está en N, puede salir del `while` y terminar.

El LTS. Un estado es una terna $(pc_1, pc_2, turn)$, con $pc_i \in \{\mathrm{N}, \mathrm{W}, \mathrm{C}\}$. Cada thread tiene tres acciones. $pide_i$ va de N a W y se puede hacer siempre. $entra_i$ va de W a C y solo se puede hacer si $turn = i$. $sale_i$ va de C a N y pone $turn = j$. Partiendo de $(\mathrm{N}, \mathrm{N}, 1)$ quedan 12 estados alcanzables. Arriba están los seis con $turn = 1$ y abajo los seis con $turn = 2$:



Notemos que no hay estados con $(\mathrm{C}, \cdot, 2)$ ni con $(\cdot, \mathrm{C}, 1)$. Para entrar a C hace falta tener el turno, y el turno solo cambia al salir.

1. $\Box\neg(critical_1 \wedge critical_2)$: vale. Es safety, así que alcanza con recorrer los estados alcanzables. En ninguno de los 12 están los dos en C. De hecho no podría haberlo, porque en C el thread i siempre tiene $turn = i$, y `turn` no puede valer 1 y 2 a la vez.

2. $\Box(wait_1 \to \Diamond critical_1)$: vale, incluso sin fair choice. Es liveness, así que buscamos un lazo que pase por un estado con $wait_1$ y después no llegue nunca a $critical_1$. Mientras el thread 1 está en W, lo único que puede hacer es $entra_1$, y eso lo lleva a C. Con lo cual un lazo así tiene que quedarse en los estados con el thread 1 en W, moviendo solo al thread 2. Miremos qué caminos hay entre esos estados sin usar $entra_1$:

$$(\mathrm{W}, \mathrm{N}, 2) \xrightarrow{pide_2} (\mathrm{W}, \mathrm{W}, 2) \xrightarrow{entra_2} (\mathrm{W}, \mathrm{C}, 2) \xrightarrow{sale_2} (\mathrm{W}, \mathrm{N}, 1) \xrightarrow{pide_2} (\mathrm{W}, \mathrm{W}, 1)$$

No hay ningún ciclo, y $(\mathrm{W}, \mathrm{W}, 1)$ tiene salida (justamente $entra_1$). Entonces no hay lazo que evite lo bueno, y la propiedad vale para todas las ejecuciones, justas o no. El thread 2 puede pasar a lo

sumo una vez antes que el thread 1, porque al salir le deja el turno, y la próxima vez se bloquea en el `await`.

Comparemos con el canal como lock. Ahí la goroutine 2 podía entrar y salir para siempre, y hacía falta fair choice para descartar esa ejecución. Acá el protocolo mismo obliga a alternar, con lo cual no necesitamos ninguna hipótesis sobre el scheduler.

3. Si el thread 2 puede terminar: deja de valer. Agregamos una acción fin_2 que lleva al thread 2 de N a un estado `fin`, desde el que no hace nada más. Miremos esta ejecución:

$$(\mathrm{N},\mathrm{N},1) \xrightarrow{\mathit{pide}_1} (\mathrm{W},\mathrm{N},1) \xrightarrow{\mathit{entra}_1} (\mathrm{C},\mathrm{N},1) \xrightarrow{\mathit{sale}_1} (\mathrm{N},\mathrm{N},2) \xrightarrow{\mathit{fin}_2} (\mathrm{N},\texttt{fin},2) \xrightarrow{\mathit{pide}_1} (\mathrm{W},\texttt{fin},2)$$

El último estado no tiene salidas. El thread 2 terminó, y el thread 1 está esperando un turno que nadie le va a dar. Por la convención de las ejecuciones que terminan, la ejecución se queda en $(\mathrm{W},\texttt{fin},2)$ para siempre. Es un lazo cuya vuelta es un ciclo de largo uno. Desde la posición 6 vale wait_1 y $\mathit{critical}_1$ no vuelve a valer.

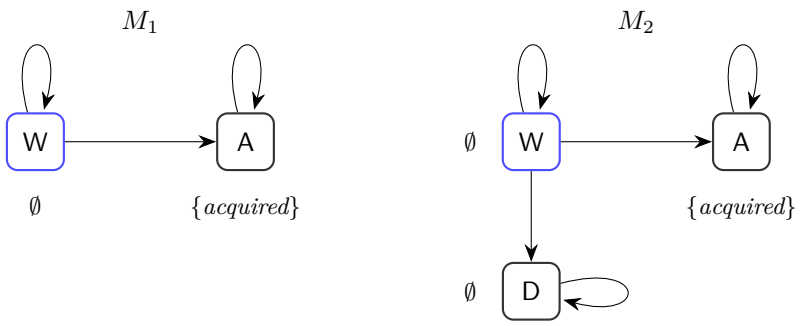
¿Lo arregla fair choice? No. Fair choice obliga a tomar las transiciones salientes de los estados que se visitan infinitas veces, pero $(\mathrm{W},\texttt{fin},2)$ no tiene ninguna. Este contraejemplo no depende del scheduler, sino del protocolo. La alternancia estricta necesita que el otro thread siga entrando, y si deja de hacerlo, el que espera se queda afuera para siempre. Notemos que la propiedad 1 sigue valiendo, porque agregar fin_2 no crea estados con los dos en C.

Distinto sería que `otras_cosas()` pudiera tardar todo lo que quiera, sin terminar. Eso lo modelamos con un lazo $\mathit{trabaja}_2$ sobre N. Sin fair choice la propiedad 2 tampoco vale, porque desde $(\mathrm{W},\mathrm{N},2)$ el thread 2 puede hacer $\mathit{trabaja}_2$ para siempre. Pero bajo fair choice ese lazo no es justo. Visita $(\mathrm{W},\mathrm{N},2)$ infinitas veces y nunca toma pide_2. Con lo cual, para fair choice, tardar no es lo mismo que terminar.

3.7. Lo que LTL no puede expresar

Hay propiedades razonables que LTL no puede expresar. Ahora, decir "no puedo escribir la fórmula" no prueba nada, así que veamos cómo se demuestra.

Pensemos en un thread que espera un lock, con la proposición *acquired*, que vale cuando ya lo consiguió. Mientras espera, otros threads pueden tomar y soltar el lock todas las veces que quieran, con lo cual el thread puede esperar indefinidamente. Comparemos dos modelos. En el segundo, además, el thread que tiene el lock puede morir sin soltarlo, y entonces el que espera queda bloqueado para siempre:



La propiedad que nos interesa es *en todo momento, todavía es posible que el thread consiga el lock*. M_1 la cumple, porque desde W y desde A se puede llegar a A. M_2 no la cumple, porque desde D no se puede llegar nunca. ¿Qué fórmula de LTL la expresa?

Ninguna. La demostración tiene dos pasos.

Paso 1: M_1 y M_2 tienen las mismas trazas. Las trazas de M_1 son $\emptyset^\omega$ (si se queda en W para siempre) y $\emptyset^n \{acquired\}^\omega$ con $n \geq 1$ (si en algún momento pasa a A). En M_2 aparecen las mismas, y las únicas nuevas son las que pasan por D. Pero esas son $\emptyset^\omega$, que ya estaba en M_1.

Paso 2: entonces satisfacen las mismas fórmulas. Recordemos que un LTS satisface φ si todas sus trazas la satisfacen. Si dos LTS tienen las mismas trazas, para cualquier φ los dos la satisfacen o los dos no. Si existiera una fórmula que expresa la propiedad, M_1 la satisfaría y M_2 no, lo cual es imposible.

Por qué pasa. LTL mira las ejecuciones *de a una*, con lo cual un sistema queda reducido al conjunto de sus trazas. "Es posible" no habla de una ejecución, sino de las *alternativas* que hay en un estado. En la traza $\emptyset^\omega$ no queda registro de por qué esperó el thread. Puede ser que otros le ganaran el lock (como en M_1) o que el lock quedara tomado para siempre (como en M_2). La diferencia está en la *ramificación* del LTS, y LTL no la ve.

3.8. Otras lógicas

CTL. La lógica de árbol computacional (CTL) interpreta las fórmulas sobre el árbol de todas las ejecuciones, y agrega cuantificadores sobre caminos. **A** es "para todos los caminos desde acá", y **E** es "existe un camino desde acá". En CTL cada operador temporal va pegado a uno de estos. La propiedad de recién se escribe

$$\mathbf{AG}\,\mathbf{EF}\,acquired$$

Es decir, "en todos los caminos, siempre, existe un camino en el que eventualmente el thread consigue el lock". M_1 la cumple y M_2 no, porque en D falla **EF** *acquired*.

Ahora, que CTL pueda expresar esto no quiere decir que sea mejor. Hay cosas de LTL que CTL no puede expresar, como $\Diamond\Box p$ ("a partir de algún momento p vale para siempre"). Ninguna de las dos lógicas contiene a la otra. CTL* las contiene a las dos, porque permite combinar libremente cuantificadores de caminos y operadores temporales.

Un comentario sobre bisimulación. Vimos que mirar solo las trazas no alcanza para distinguir a M_1 de M_2. ¿Hay una forma de comparar dos LTS que sí vea la ramificación? Sí, y se llama **bisimulación**. La idea es pensarlo como un juego entre dos sistemas. Cada vez que uno da un paso, el otro tiene que poder imitarlo, llegando a un estado con las mismas proposiciones, y desde ahí el juego sigue. Si el segundo siempre puede imitar al primero, y al revés, los dos son *bisimilares*.

M_1 y M_2 no lo son. Si M_2 va de W a D, M_1 solo puede imitarlo quedándose en W (que también tiene $\emptyset$). Pero después M_1 puede ir a A, y M_2, desde D, no lo puede imitar. Esa es justamente la diferencia que LTL no veía.

No vamos a profundizar, pero vale la pena saber dos cosas. Bisimulación es más fina que tener las mismas trazas: si dos LTS son bisimilares tienen las mismas trazas, pero no al revés. Y dos LTS bisimilares satisfacen las mismas fórmulas, tanto de LTL como de CTL. Por eso se usa, por ejemplo, para achicar un LTS juntando estados que se comportan igual, o para comparar un modelo con su especificación.

Y hay más. Según lo que se quiera decir, hay otras lógicas:

- LTL con *pasado*, con operadores como "alguna vez antes". No agrega expresividad, porque todo lo que se dice con pasado se puede decir sin él. Lo que cambia es la comodidad. "Si se responde, antes se pidió" con pasado es casi la frase. Sin pasado hay que darla vuelta: $\neg \mathit{respond}\, \mathbf{W}\, \mathit{request}$ (no se responde hasta que se pide).
- Lógicas de *tiempo real*, como MTL o TCTL, para decir cosas como "responde en menos de 5 milisegundos".
- Lógicas *probabilísticas*, como PCTL, para decir cosas como "con probabilidad al menos 0,99, el mensaje llega".
- El μ-cálculo, que es más expresivo que CTL*, y que usan herramientas como mCRL2.

Cada herramienta elige la suya. Por ejemplo, SPIN verifica LTL, NuSMV verifica CTL y LTL, UPPAAL verifica propiedades de tiempo real y PRISM, propiedades probabilísticas.

3.9. Cómo se hace en la práctica

Hasta acá buscamos los contraejemplos a mano, mirando el dibujo. Ahora, ¿cómo lo hace una herramienta como SPIN, NuSMV o LTSA? La idea es la misma, pero hecha de forma sistemática.

Primero, la herramienta niega la fórmula. Que todas las ejecuciones cumplan φ es lo mismo que no exista ninguna que cumpla $\neg\varphi$. Con lo cual lo que busca es un contraejemplo.

Después, pasa $\neg\varphi$ a un autómata (de Büchi) que acepta exactamente las trazas que la cumplen. Para armarlo se desarma la fórmula en lo que tiene que valer ahora y lo que queda pendiente para después. Por ejemplo, $\Diamond p \equiv p \vee \bigcirc \Diamond p$ (o vale p ahora, o lo seguimos esperando). Cada estado del autómata es una de esas obligaciones pendientes.

Luego recorre el LTS y el autómata a la vez. Los estados son pares (un estado del LTS y uno del autómata), y en cada paso avanzan los dos. Los caminos de este producto son justamente las ejecuciones del LTS que cumplen $\neg\varphi$.

Por último, busca un lazo. Recordemos que el grafo es finito, con lo cual una ejecución infinita termina dando vueltas en un ciclo. Hay contraejemplo si hay un lazo que pasa infinitas veces por un estado de aceptación del autómata. Es el mismo lazo de la tabla de safety y liveness. La diferencia es que acá el autómata dice qué lazos sirven, para cualquier fórmula.

¿Y si queremos demostrar que φ vale? Es el mismo algoritmo. La herramienta recorre todo el producto, y si no encuentra ningún lazo, φ vale. Refutar es barato (corta en el primer lazo), pero demostrar obliga a recorrer todo. Por eso las herramientas no arman el producto de antemano, sino que lo van generando a medida que lo recorren, y evitan explorar todos los órdenes de acciones independientes (que llevan al mismo estado). NuSMV va por otro lado. No recorre estado por estado, sino que trabaja con conjuntos de estados representados con fórmulas booleanas.

Ahora, todo esto termina porque el LTS es finito. Si el sistema tiene infinitos estados (un contador sin cota, o una cantidad arbitraria de threads), la búsqueda puede no terminar. En ese caso se puede acotar, es decir, verificar para 2, 3 o 4 threads. Eso no es una prueba general, aunque si aparece un contraejemplo, es real. También se puede abstraer, armando un modelo finito que tenga al menos todos los comportamientos del real (si la propiedad vale ahí, vale en el sistema). Y si no, queda demostrarlo a mano, o con herramientas como TLA+ o Dafny. Para safety se busca un

invariante inductivo (algo que vale al inicio y que toda transición preserva, como "en C el thread i tiene $turn = i$" en el ejercicio de `turn`). Para liveness, una medida que decrece mientras no pasa lo bueno.

Como moraleja, al usar una herramienta conviene fijarse en dos convenciones. Cómo trata los estados sin salida (NuSMV, por ejemplo, pide agregar el autoloop a mano) y si supone fair choice o hay que pedirlo. Si no son las mismas que usamos a mano, los resultados pueden no coincidir.